Платформено-независими програмни езици

Лекция 9

Тестване и логване

Maven, модулни тестове с JUnit 6, параметризирани тестове, mock обекти, логване с java.util.logging и SLF4J

проф. д-р Даниела Гоцева
Учебна 2026/2027 г.



Съдържание на лекцията

1. Защо тестваме?
2. Maven накратко
3. Модулни тестове с JUnit 6
4. По-добри тестове
5. Mock обекти
6. Логване

01

Защо тестваме?

Качество, увереност при промени, документация

Цената на грешките

- Грешка, открита при писането на кода – струва минути
- Открита при тестване – часове
- Открита при клиента – дни, пари, репутация
- Тестовете дават **увереност**, че промяната не е счупила нещо друго
- Тестовете са **жива документация** – показват как се използва кодът
- Код, генериран от ИИ, също трябва да се тества – особено той!

Видове тестове

Вид	Какво проверява	Скорост	Брой
Модулни (unit)	един клас или метод, изолирано	милисекунди	много
Интеграционни	няколко компонента заедно: БД, файлове, мрежа	секунди	по-малко
Системни (end-to-end)	цялото приложение като потребител	минути	малко
Приемни (acceptance)	изискванията на клиента	различна	малко

Пирамида на тестовете

- **Основа** – много бързи модулни тестове
- **Среда** – по-малко интеграционни тестове
- **Връх** – малко бавни тестове на цялата система
- Защо пирамида? Модулните тестове са евтини, бързи и точно посочват проблема
- Автоматизирани тестове се пускат при **всяка промяна** – ръчното тестване не мащабира

Какво е добър модулен тест?

- **Бърз** – стотици тестове за секунди
- **Независим** – не зависи от други тестове и от реда им
- **Повторим** – един и същ резултат всеки път, без мрежа и случайност
- **Самопроверяващ се** – сам казва „успех“ или „провал“
- **Навременен** – пише се заедно с кода (или преди него)
- Принципите **FIRST**: Fast, Independent, Repeatable, Self-validating, Timely

Тестове в Python и в Java

Python

- `unittest` (вграден) или `pytest`
- `def test_deposit():`
- `assert acc.balance == 100`
- `with pytest.raises(ValueError):`
- `@pytest.mark.parametrize`

Java

- JUnit (външна библиотека, чрез Maven)
- `@Test void deposit()`
- `assertEquals(100, acc.getBalance())`
- `assertThrows(...)`
- `@ParameterizedTest`

02

Maven накратко

Изграждане на проекта и управление на зависимостите

Защо инструмент за изграждане?

- Реалните проекти използват **външни библиотеки**: JUnit, Jackson, SLF4J…
- Ръчното сваляне на JAR файлове е бавно и води до конфликти на версии
- **Maven** (и **Gradle**):
  - сваля библиотеките автоматично от **Maven Central**
  - компилира, тества и пакетира с една команда
  - стандартна структура на проекта
  - работи еднакво в Eclipse, в терминала и на сървър

Стандартна структура на Maven проект

Път	Съдържание
`pom.xml`	описание на проекта и зависимостите
`src/main/java`	изходният код на приложението
`src/main/resources`	ресурси: конфигурация, `logback.xml`
`src/test/java`	тестовете
`src/test/resources`	ресурси за тестовете
`target/`	резултат от изграждането: `.class`, `.jar`, отчети

Файлът pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>
    <groupId>bg.tu.pik3</groupId>          <!-- организация -->
    <artifactId>bank</artifactId>          <!-- име на проекта -->
    <version>1.0</version>
    <properties>
        <maven.compiler.release>21</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>6.1.3</version>
            <scope>test</scope>            <!-- само за тестовете -->
        </dependency>
    </dependencies>
</project>
```

Основни команди на Maven

Команда	Действие
`mvn compile`	компилира кода от `src/main/java`
`mvn test`	компилира и пуска тестовете
`mvn package`	създава `.jar` в `target/`
`mvn clean`	изтрива `target/`
`mvn clean install`	всичко + инсталира в локалното хранилище
`mvn dependency:tree`	показва дървото на зависимостите

Maven в Eclipse

- Eclipse има вградена поддръжка на Maven (**m2e**)
- Нов проект: File → New → **Maven Project** → *Create a simple project*
- Съществуващ проект: File → Import → **Existing Maven Projects**
- След промяна на `pom.xml`: десен бутон → Maven → **Update Project** (Alt+F5)
- Стартиране на цели: Run As → **Maven test** / **Maven build…**
- Тестове: десен бутон върху клас → Run As → **JUnit Test**

JAR файлове и пакети

- **JAR** (Java ARchive) – ZIP архив с `.class` файлове и ресурси
- `MANIFEST.MF` – може да указва главния клас
- Изпълним JAR: `java -jar app.jar`
- Библиотеките се разпространяват като JAR файлове в Maven Central
- Координати на библиотека: `groupId:artifactId:version`
  – `com.fasterxml.jackson.core:jackson-databind:2.20.0`
- **Gradle** – алтернатива с описание на Kotlin/Groovy вместо XML

Модулна система (Java 9+)

- **Модул** – група от пакети с явно описани зависимости
- Файл `module-info.java` в корена на изходния код
- `requires` – от кои модули зависим
- `exports` – кои пакети са видими отвън
- Силно капсулиране на ниво библиотека; JDK също е разделен на модули

```
// src/main/java/module-info.java
module bg.tu.pik3.bank {
    requires java.net.http;
    requires org.slf4j;

    exports bg.tu.pik3.bank.api;
    // bg.tu.pik3.bank.internal
    // остава скрит
}
```

03

Модулни тестове с JUnit 6

Първи тест, проверки, жизнен цикъл

Класът, който ще тестваме

```
package bg.tu.pik3.bank;

public class Account {
    private double balance;
    public double getBalance() { return balance; }
    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Сумата трябва да е > 0");
        }
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount > balance) {
            throw new IllegalStateException("Недостатъчна наличност");
        }
        balance -= amount;
    }
}
```

Първи тест

```java
package bg.tu.pik3.bank;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class AccountTest {

    @Test
    void depositIncreasesBalance() {
        Account acc = new Account();          // Arrange – подготовка

        acc.deposit(100);                     // Act – действие

        assertEquals(100.0, acc.getBalance(), 0.001);   // Assert – проверка
    }
}
```

JUnit 6 запазва API на JUnit 5 (Jupiter) – примерите работят и с двете версии



Arrange – Act – Assert

Всеки тест има три ясни части: **подготовка** на обектите, **действие**, което тестваме, и **проверка** на резултата. Един тест – едно поведение.

Основни проверки (Assertions)

Метод	Проверява
`assertEquals(expected, actual)`	равенство (за `double` – с точност `delta`)
`assertNotEquals(a, b)`	неравенство
`assertTrue(cond)`, `assertFalse(cond)`	логическо условие
`assertNull(x)`, `assertNotNull(x)`	`null` / не `null`
`assertSame(a, b)`	един и същ обект
`assertArrayEquals(a, b)`	еднакви масиви
`assertThrows(Type.class, () -> ...)`	хвърля изключение от типа
`assertAll(...)`	няколко проверки – отчита всички провали

Тестване на изключения

- `assertThrows` – изпълнява ламбдата и очаква изключение
- Връща изключението – може да проверим съобщението
- Ако **не** се хвърли – тестът пада

```
@Test
void depositRejectsNegativeAmount() {
    Account acc = new Account();

    var ex = assertThrows(
            IllegalArgumentException.class,
            () -> acc.deposit(-5));

    assertTrue(ex.getMessage()
                 .contains("> 0"));
}
```

Състоянието след изключение

- Проверяваме и че обектът **не се е променил** след неуспешна операция
- Така тестът хваща частично изпълнени промени

```
@Test
void cannotWithdrawMoreThanBalance() {
    Account acc = new Account();
    acc.deposit(50);

    assertThrows(IllegalStateException.class,
            () -> acc.withdraw(100));

    assertEquals(50, acc.getBalance(),
            0.001);
}
```

Жизнен цикъл на тестовете

Анотация	Кога се изпълнява	Типична употреба
`@BeforeAll`	веднъж преди всички тестове (static)	скъпа подготовка: връзка към тестова БД
`@BeforeEach`	преди **всеки** тест	нов обект за теста
`@Test`	самият тест	–
`@AfterEach`	след всеки тест	почистване: временни файлове
`@AfterAll`	веднъж след всички (static)	освобождаване на ресурси

@BeforeEach и @DisplayName

- Всеки тест получава **нов** обект – тестовете са независими
- `@DisplayName` – четимо описание в отчета
- В Eclipse резултатът се вижда в изгледа **JUnit**: зелено / червено

```
@DisplayName("Банкова сметка")
class AccountTest {
    private Account acc;

    @BeforeEach
    void setUp() {
        acc = new Account();
        acc.deposit(100);
    }

    @Test
    @DisplayName("Теглене намалява баланса")
    void withdraw() {
        acc.withdraw(30);
        assertEquals(70, acc.getBalance(), 1e-9);
    }
}
```

assertAll – групови проверки

- Обикновено първата неуспешна проверка спира теста
- `assertAll` изпълнява **всички** и отчита всички провали
- Полезно за проверка на няколко полета на обект

```
@Test
void parsesStudentFromCsv() {
    Student s = Student.parse("Ана, 101, 5.5");

    assertAll("студент",
        () -> assertEquals("Ана", s.name()),
        () -> assertEquals(101, s.fn()),
        () -> assertEquals(5.5, s.grade(), 1e-9)
    );
}
```

Пример: тестове за Fraction

```
class FractionTest {
    @Test
    void reducesToLowestTerms() {
        assertEquals("1/2",
            new Fraction(4, 8).toString());
    }

    @Test
    void keepsSignInNumerator() {
        assertEquals("-1/3",
            new Fraction(2, -6).toString());
    }

    @Test
    void rejectsZeroDenominator() {
        assertThrows(ArithmeticException.class,
            () -> new Fraction(1, 0));
    }
}
```

Добри имена на тестове

- Името описва **поведението**, а не метода:
  - лошо: `test1()`, `testWithdraw()`
  - добре: `withdrawFailsWhenBalanceIsInsufficient()`
- Шаблон: *метод_условие_очакван резултат*
- Тестовете се четат като **спецификация** на класа
- Когато тест падне, името трябва да казва какво е счупено
- `@DisplayName` – за описание на български

Тестове в Eclipse

- Тестов клас: десен бутон върху класа → New → **JUnit Test Case**
- Стартиране: Run As → **JUnit Test** (Alt+Shift+X, T)
- Изгледът **JUnit**: зелена лента – всичко минава, червена – има провал
- Двоен клик върху паднал тест → отваря реда с проблема
- **Failure Trace** – очаквано срещу получено
- Повторно пускане само на падналите тестове – бутон *Rerun Failed First*

04

По-добри тестове

Параметризирани тестове, гранични стойности, покритие

Параметризирани тестове

- Един тест – много набори от данни
- `@ParameterizedTest` вместо `@Test`
- `@ValueSource` – прост списък от стойности
- Нужна е зависимостта `junit-jupiter-params` (включена в `junit-jupiter`)

```java
@ParameterizedTest
@ValueSource(ints = {2, 3, 5, 7, 11, 97})
void primesAreRecognized(int n) {
    assertTrue(MathUtils.isPrime(n));
}

@ParameterizedTest
@ValueSource(ints = {-1, 0, 1, 4, 100})
void nonPrimesAreRejected(int n) {
    assertFalse(MathUtils.isPrime(n));
}
```

@CsvSource – вход и очакван резултат

- Всеки ред е отделен тестов случай
- Стойностите се преобразуват към типовете на параметрите
- `name` – описание на всеки случай в отчета

```
@ParameterizedTest(name = "{0} точки -> {1}")
@CsvSource({
    "0,   2",
    "49,  2",
    "50,  3",
    "64,  3",
    "65,  4",
    "90,  6",
    "100, 6"
})
void gradeForPoints(int points, int expected) {
    assertEquals(expected,
            Grading.grade(points));
}
```

@MethodSource – сложни данни

- Данните идват от статичен метод
- Подходящо за обекти и записи
- `Arguments.of(...)` – набор от аргументи

```
static Stream<Arguments> fractions() {
    return Stream.of(
        Arguments.of(new Fraction(1, 2),
                     new Fraction(1, 3), "5/6"),
        Arguments.of(new Fraction(1, 2),
                     new Fraction(-1, 2), "0/1"),
        Arguments.of(new Fraction(2, 4),
                     new Fraction(1, 4), "3/4"));
}

@ParameterizedTest
@MethodSource("fractions")
void addition(Fraction a, Fraction b,
              String expected) {
    assertEquals(expected,
            a.plus(b).toString());
}
```

Какво да тестваме? Гранични стойности

- Грешките се крият по **границите**
- За условие `50 <= points <= 100` тестваме: 49, 50, 51, 99, 100, 101
- Специални стойности:
  - `0`, отрицателни числа, `Integer.MAX_VALUE`
  - празен низ, `null`, низ с интервали
  - празна колекция, един елемент, много елементи
- **Класове на еквивалентност** – по един представител от всяка група входни данни
- Тествайте и **грешните** входове – не само „щастливия път“

@Nested, @Disabled и предположения

- `@Nested` – групиране на свързани тестове във вътрешен клас
- Отчетът в Eclipse показва дървовидна структура
- Удобно за различни начални състояния

```
class AccountTest {

    @Nested
    @DisplayName("Празна сметка")
    class Empty {
        @Test
        void hasZeroBalance() {
            assertEquals(0,
                new Account().getBalance());
        }
    }

    @Nested
    @DisplayName("Сметка със 100 лв.")
    class WithMoney { /* ... */ }
}
```

Изключени тестове и предположения

- `@Disabled` – временно изключен тест; винаги пишете причината
- `assumeTrue` – тестът се **пропуска** (не пада), ако условието не е изпълнено
- `@Tag` – групиране, напр. бавни тестове

```
@Test
@Disabled("Чака поправка на #42")
void transferBetweenAccounts() { }

@Test
void onlyOnLinux() {
    assumeTrue(System.getProperty("os.name")
                          .contains("Linux"));
    // ...
}

@Test
@Tag("slow")
void importsLargeFile() { }
```

Временни файлове в тестовете: @TempDir

- JUnit създава празна временна папка за теста
- Изтрива я автоматично след теста
- Тестовете с файлове остават независими

```
@Test
void savesReport(@TempDir Path dir)
        throws IOException {
    Path file = dir.resolve("report.txt");

    new ReportWriter().save(file,
            List.of("Ана 6", "Боян 5"));

    assertEquals(List.of("Ана 6", "Боян 5"),
            Files.readAllLines(file));
}
```

Покритие на кода (code coverage)

- Какъв процент от редовете/разклоненията се изпълняват от тестовете
- Инструменти: **JaCoCo** (Maven плъгин), **EclEmma** в Eclipse (Coverage As → JUnit Test)
- Зелено – изпълнен ред, червено – неизпълнен, жълто – частично
- Високото покритие **не гарантира** правилност – проверките са важни
- Ориентир: 70–80 % за бизнес логиката
- Ниското покритие показва **къде** липсват тестове

Бързи тестове: assertTimeout

- Тест, който продължава твърде дълго, пада
- Пазим се от безкрайни цикли и бавни алгоритми
- `@Timeout` – ограничение за целия тест

```
@Test
void sortIsFast() {
    int[] data = new Random(1)
            .ints(1_000_000).toArray();

    assertTimeout(Duration.ofSeconds(1),
            () -> Arrays.sort(data));
}

@Test
@Timeout(2)      // секунди
void importFinishes() {
    importer.run();
}
```

Разработка, водена от тестове (TDD)

- Цикъл **червено – зелено – рефакториране**:
  - 1. Напиши тест за ново поведение – **пада** (червено)
  - 2. Напиши минималния код, с който тестът минава (зелено)
  - 3. Подобри кода, като тестовете остават зелени
- Тестът е **първият клиент** на кода – помага за добър дизайн
- Работи отлично с ИИ асистенти: тестовете описват **какво**, ИИ предлага **как**, тестовете **проверяват**

Тестване на код, генериран от ИИ

- ИИ асистентите пишат код, който **изглежда** правилен
- Типични проблеми: гранични случаи, грешна обработка на `null`, остарели API
- Подход:
  - първо **вие** пишете тестовете – те описват изискванията
  - ИИ предлага реализация
  - тестовете решават дали е приета
- Можете да поискате от ИИ и тестове – но **ги прочетете**: тест, който нищо не проверява, винаги минава
- Отговорността за кода остава ваша

05

Mock обекти

Изолиране на кода от зависимостите му

Проблемът със зависимостите

- Класът често зависи от други: база данни, мрежа, имейл, ИИ услуга
- Модулният тест **не трябва** да ги използва:
  - бавни, нестабилни, струват пари
  - трудно се възпроизвеждат грешки
- Решение: заместваме зависимостта с **тестов двойник** (test double)
- Условие: класът получава зависимостта отвън – чрез конструктор и **интерфейс**

Видове тестови двойници

Вид	Описание
Dummy	подава се, но не се използва
Stub	връща предварително зададени отговори
Fake	опростена работеща реализация (напр. в паметта)
Mock	записва извикванията – проверяваме дали и как е извикан
Spy	реален обект, чиито извиквания се следят

Ръчен fake чрез интерфейс

- Зависимостта е **интерфейс**
- В продукцията – реална реализация (HTTP заявка към банка)
- В теста – ламбда с фиксиран отговор

```
interface RateService {
    double rate(String from, String to);
}

record Converter(RateService rates) {
    double convert(double amt,
                   String from, String to) {
        return amt * rates.rate(from, to);
    }
}

@Test
void convertsWithGivenRate() {
    RateService fake = (f, t) -> 1.95583;
    var c = new Converter(fake);
    assertEquals(195.583,
        c.convert(100, "EUR", "BGN"), 1e-6);
}
```

Mockito

- Най-популярната библиотека за mock обекти
- `mock(Клас)` – създава двойник
- `when(...).thenReturn(...)` – задава поведение
- `verify(...)` – проверява извикване

```
import static org.mockito.Mockito.*;

@Test
void sendsEmailOnLargeWithdrawal() {
    Mailer mailer = mock(Mailer.class);
    var acc = new Account(mailer);
    acc.deposit(5000);

    acc.withdraw(3000);

    verify(mailer).send(
            contains("3000"));
    verifyNoMoreInteractions(mailer);
}
```

06

Логване

Записване на събитията в работещото приложение

System.out.println или логер?

System.out.println

- няма нива на важност
- няма време, нишка, клас
- не може да се изключи без промяна на кода
- само към конзолата
- смесва се с нормалния изход

Логер

- нива: грешка, предупреждение, информация, детайли
- автоматично време, клас, нишка
- настройва се от конфигурационен файл
- към конзола, файл, сървър за логове
- стандарт в професионалните приложения

Нива на логване

java.util.logging	SLF4J / Logback	Кога
`SEVERE`	`ERROR`	грешка – операцията е неуспешна
`WARNING`	`WARN`	нещо необичайно, но работим
`INFO`	`INFO`	важни събития: старт, вход на потребител
`CONFIG`	–	конфигурационни съобщения
`FINE`, `FINER`	`DEBUG`	подробности за отстраняване на грешки
`FINEST`	`TRACE`	най-подробно проследяване

java.util.logging (JUL)

- Вграден в JDK – без зависимости
- Логер за всеки клас: `Logger.getLogger(...)`
- По подразбиране показва `INFO` и по-високо

```
import java.util.logging.Level;
import java.util.logging.Logger;

public class Account {
    private static final Logger LOG =
        Logger.getLogger(Account.class.getName());

    public void withdraw(double amount) {
        LOG.fine("Теглене: " + amount);
        if (amount > balance) {
            LOG.warning("Недостатъчна наличност");
            throw new IllegalStateException();
        }
        balance -= amount;
        LOG.info(() -> "Нов баланс: " + balance);
    }
}
```

Логване на изключения

- Записваме и **контекста**, и самото изключение
- Стекът се записва в лога – не губим информация
- Не логвайте и не хвърляйте едно и също изключение на всяко ниво

```
try {
    importFile(path);
} catch (IOException e) {
    LOG.log(Level.SEVERE,
            "Импортът на " + path
            + " е неуспешен", e);
}

// В лога:
// SEVERE: Импортът на data.csv е неуспешен
// java.nio.file.NoSuchFileException: data.csv
//     at ...
```

Настройка на java.util.logging

```
# logging.properties
handlers = java.util.logging.ConsoleHandler, java.util.logging.FileHandler
.level = INFO

bg.tu.pik3.level = FINE                          # по-подробно за нашия пакет

java.util.logging.ConsoleHandler.level = INFO
java.util.logging.FileHandler.pattern = logs/app-%u.log
java.util.logging.FileHandler.formatter = java.util.logging.SimpleFormatter
java.util.logging.SimpleFormatter.format = %1$tF %1$tT %4$-7s %3$s - %5$s%6$s%n
```

Стартиране: `java -Djava.util.logging.config.file=logging.properties -jar app.jar`

SLF4J и Logback

- **SLF4J** – фасада: единен API за логване
- **Logback** – реализация (също Log4j 2, JUL)
- Кодът зависи само от SLF4J – реализацията се сменя в `pom.xml`
- Използва се от повечето библиотеки и от Spring Boot
- Зависимост: `ch.qos.logback:logback-classic` (включва SLF4J API)

Логване със SLF4J

- `{}` – параметри; низът се сглобява **само ако** нивото е включено
- Изключението се подава **последно** – записва се със стека
- Не слепвайте низове с `+` в логовете

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class OrderService {
    private static final Logger log =
        LoggerFactory.getLogger(OrderService.class);

    public void place(Order o) {
        log.info("Нова поръчка {} за {} лв.",
                 o.id(), o.total());
        try {
            payment.charge(o);
        } catch (PaymentException e) {
            log.error("Плащането за {} е неуспешно",
                      o.id(), e);
        }
        log.debug("Детайли: {}", o);
    }
}
```

Конфигурация на Logback

```
<!-- src/main/resources/logback.xml -->
<configuration>
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{HH:mm:ss} %-5level [%thread] %logger{20} - %msg%n</pattern>
        </encoder>
    </appender>

    <appender name="FILE" class="ch.qos.logback.core.FileAppender">
        <file>logs/app.log</file>
        <encoder><pattern>%d %-5level %logger - %msg%n</pattern></encoder>
    </appender>

    <logger name="bg.tu.pik3" level="DEBUG"/>
    <root level="INFO">
        <appender-ref ref="CONSOLE"/>
        <appender-ref ref="FILE"/>
    </root>
</configuration>
```

Какво да логваме и какво не

Логвайте

- старт и спиране на приложението
- важни бизнес събития: поръчка, плащане
- грешки – с изключението и контекста
- извиквания към външни системи и времето им
- предупреждения за необичайни ситуации

Никога не логвайте

- пароли, API ключове, токени
- ЕГН, номера на карти, лични данни
- огромни обекти на всяка стъпка
- едно и също изключение на няколко нива
- в тесни цикли на ниво `INFO`

Непрекъсната интеграция (CI)

- Тестовете се пускат **автоматично** при всяко качване в Git
- GitHub Actions, GitLab CI, Jenkins
- Червен тест → промяната не се приема
- Пример за GitHub Actions: файл `.github/workflows/build.yml`
  - стъпки: изтегли кода → инсталирай JDK 21 → `mvn -B test`
- Логовете от CI показват кой тест и защо е паднал

Обобщение

- Автоматизираните тестове дават увереност при промени; пирамида на тестовете
- **Maven** – структура на проекта, зависимости, `mvn test`, `mvn package`
- **JUnit 6**: `@Test`, `assertEquals`, `assertThrows`, `assertAll`
- Arrange – Act – Assert; `@BeforeEach` за независими тестове
- Параметризирани тестове: `@ValueSource`, `@CsvSource`, `@MethodSource`
- Гранични стойности, покритие, TDD
- Тестови двойници: fake чрез интерфейс, **Mockito**
- Логване: нива, `java.util.logging`, **SLF4J + Logback**, без лични данни

Въпроси за самопроверка

- Какво е модулен тест и какви свойства трябва да има?
- Каква е ролята на `pom.xml`? Какво означава `<scope>test</scope>`?
- Как се тества, че метод хвърля изключение?
- Кога използваме параметризирани тестове?
- Кои гранични стойности бихте тествали за `1 <= n <= 10`?
- Защо и как заместваме зависимостите с mock обекти?
- Защо логерът е по-добър от `System.out.println`? Какво не трябва да се логва?

Въпроси?

Следваща лекция: Java и изкуствен интелект

